VIT Vidyalankar Institute of Technology Accredited A+ by NAAC	DEPARTMENT OF COMPUTER ENGINEERING

PBLE Report

Semester	S.E. Semester III – Computer Engineering
Subject	Analysis of Algorithm
Subject Professor In-charge	Dr. Swapnil S. Sonawane

Roll Number	Name of Students
25102A0022	Manthan Rane

Name of the Project: Campus Shortest Route Finder – Dijkstra's Algorithm

Project Description :

This project is a menu-driven C program designed to model a college campus road network and calculate the shortest travel distances between key locations. The system represents campus locations—such as the Main Gate, Library, Computer Department, Laboratory, and Auditorium—as nodes within a weighted graph using an adjacency matrix.

The core functionality utilizes Dijkstra's Algorithm (a Greedy Method approach) to determine the most efficient paths from a user-selected source location to all other destinations on campus.

Key Features & Implementation Details:

Graph Representation: Utilizes an adjacency matrix where non-connected routes are represented by an 'INF' value and diagonal elements are 0.

Path Tracking: Maintains a parent/predecessor array to reconstruct and display the exact sequence of locations for each shortest path.

Menu Interface: Allows users to input custom graph data, display the adjacency matrix, set a starting location, execute the algorithm, and view the resulting distances and paths.

Performance: The implemented algorithm operates with a time complexity of O(V^2), which is standard for an array-based implementation of Dijkstra's Algorithm.

Project Code :

```
#include <stdio.h>

#include <limits.h>
```

```
#include <stdbool.h>
#include <string.h>

#define MAX 10
#define INF 9999

int graph[MAX][MAX];
char locations[MAX][50];
int dist[MAX];
int parent[MAX];
int num_locations = 5;
int source = -1;
bool is_calculated = false;

int minDistance(int dist[], bool sptSet[]) {
   int min = INF, min_index;
   for (int v = 0; v < num_locations; v++) {
      if (sptSet[v] == false && dist[v] <= min) {
         min = dist[v];
         min_index = v;
      }
   }
   return min_index;
}

void printPath(int current_vertex) {
```

```
    if (parent[current_vertex] == -1) {
        printf("%s", locations[current_vertex]);
        return;
    }
    printPath(parent[current_vertex]);
    printf(" -> %s", locations[current_vertex]);
}

void dijkstra() {
    if (source == -1) {
        printf("Please select a source location first.\n");
        return;
    }

    bool sptSet[MAX];

    for (int i = 0; i < num_locations; i++) {
        dist[i] = INF;
        sptSet[i] = false;
        parent[i] = -1;
    }

    dist[source] = 0;

    for (int count = 0; count < num_locations - 1; count++) {
        int u = minDistance(dist, sptSet);
```

```
        sptSet[u] = true;

        for (int v = 0; v < num_locations; v++) {
            if (!sptSet[v] && graph[u][v] && graph[u][v] != INF && dist[u] != INF
                && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
                parent[v] = u;
            }
        }
    }
    is_calculated = true;
    printf("\nShortest distances calculated successfully.\n");
}

int main() {
    int choice;

    strcpy(locations[0], "Main Gate");
    strcpy(locations[1], "Library");
    strcpy(locations[2], "Computer Department");
    strcpy(locations[3], "Laboratory");
    strcpy(locations[4], "Auditorium");

    for (int i = 0; i < MAX; i++) {
        for (int j = 0; j < MAX; j++) {
            if (i == j) graph[i][j] = 0;
```

```
        else graph[i][j] = INF;
    }
}

while (1) {
    printf("\n--- Campus Shortest Route Finder ---\n");
    printf("1. Enter Campus Graph\n");
    printf("2. Display Adjacency Matrix\n");
    printf("3. Select Source Location\n");
    printf("4. Find Shortest Distance\n");
    printf("5. Display Shortest Paths\n");
    printf("6. Display Distance from Source to All Locations\n");
    printf("7. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            printf("Enter number of locations (max %d): ", MAX);
            scanf("%d", &num_locations);
            getchar(); // clear newline
            for(int i=0; i<num_locations; i++) {
                printf("Enter name for location %d: ", i+1);
                fgets(locations[i], 50, stdin);
                locations[i][strcspn(locations[i], "\n")] = 0;
            }
```

```
            printf("Enter the adjacency matrix (%d x %d). Use %d for INF:\n", num_locations, num_locations,
INF);
            for (int i = 0; i < num_locations; i++) {
                for (int j = 0; j < num_locations; j++) {
                    scanf("%d", &graph[i][j]);
                }
            }
            is_calculated = false;
            break;

        case 2:
            printf("\nAdjacency Matrix:\n");
            for (int i = 0; i < num_locations; i++) {
                for (int j = 0; j < num_locations; j++) {
                    if (graph[i][j] == INF) printf("INF\t");
                    else printf("%d\t", graph[i][j]);
                }
                printf("\n");
            }
            break;

        case 3:
            printf("\nLocations:\n");
            for(int i=0; i<num_locations; i++) {
                printf("%d. %s\n", i+1, locations[i]);
            }
```

```
    printf("Select source location (1-%d): ", num_locations);
    scanf("%d", &source);
    source--;
    is_calculated = false;
    break;

case 4:
    dijkstra();
    break;

case 5:
    if (!is_calculated) {
        printf("Calculate shortest distances first (Option 4).\n");
        break;
    }
    printf("\nDestination\t\tShortest Distance\tShortest Path\n");
    printf(" ------------------------------------------------------------------\n");
    for (int i = 0; i < num_locations; i++) {
        if (i != source) {
            printf("%-25s %-20d ", locations[i], dist[i]);
            printPath(i);
            printf("\n");
        }
    }
    break;
```

```
            case 6:
                if (!is_calculated) {
                    printf("Calculate shortest distances first (Option 4).\n");
                    break;
                }
                printf("\nDistances from %s:\n", locations[source]);
                for (int i = 0; i < num_locations; i++) {
                    if (dist[i] == INF) {
                        printf("To %s: Unreachable\n", locations[i]);
                    } else {
                        printf("To %s: %d\n", locations[i], dist[i]);
                    }
                }
                break;

            case 7:
                return 0;

            default:
                printf("Invalid choice. Please try again.\n");
        }
    }
}
```

Output Screenshots :

```
--- Campus Shortest Route Finder ---
1. Enter Campus Graph
2. Display Adjacency Matrix
3. Select Source Location
4. Find Shortest Distance
5. Display Shortest Paths
6. Display Distance from Source to All Locations
7. Exit
Enter your choice: 1
Enter number of locations (max 10): 5
Enter name for location 1: Hostel
Enter name for location 2: Canteen
Enter name for location 3: Sports Ground
Enter name for location 4: Admin Block
Enter name for location 5: Library
Enter the adjacency matrix (5 x 5). Use 9999 for INF:
0 2 5 9999 9999
2 0 1 4 9999
5 1 0 9999 3
9999 4 9999 0 1
9999 9999 3 1 0
```

```
--- Campus Shortest Route Finder ---
1. Enter Campus Graph
2. Display Adjacency Matrix
3. Select Source Location
4. Find Shortest Distance
5. Display Shortest Paths
6. Display Distance from Source to All Locations
7. Exit
Enter your choice: 3

Locations:
1. Hostel
2. Canteen
3. Sports Ground
4. Admin Block
5. Library
Select source location (1-5): 1

--- Campus Shortest Route Finder ---
1. Enter Campus Graph
2. Display Adjacency Matrix
3. Select Source Location
4. Find Shortest Distance
5. Display Shortest Paths
6. Display Distance from Source to All Locations
7. Exit
Enter your choice: 4

Shortest distances calculated successfully.
```

```
Shortest distances calculated successfully.

--- Campus Shortest Route Finder ---
1. Enter Campus Graph
2. Display Adjacency Matrix
3. Select Source Location
4. Find Shortest Distance
5. Display Shortest Paths
6. Display Distance from Source to All Locations
7. Exit
Enter your choice: 5

Destination             Shortest Distance       Shortest Path
-------------------------------------------------------------------------
Canteen                 2                       Hostel -> Canteen
Sports Ground           3                       Hostel -> Canteen -> Sports Ground
Admin Block             6                       Hostel -> Canteen -> Admin Block
Library                 6                       Hostel -> Canteen -> Sports Ground -> Library

--- Campus Shortest Route Finder ---
1. Enter Campus Graph
2. Display Adjacency Matrix
3. Select Source Location
4. Find Shortest Distance
5. Display Shortest Paths
6. Display Distance from Source to All Locations
7. Exit
```